

# Module 3 — MCP Server

Capabilities as tools: the standard protocol for AI agents

# 🎯 Learning goals

- Understand **MCP concepts**: discovery, schemas, calling conventions.
- Build an **MCP server** with practical tools in Python.
- Compare **transports**: stdio (local) vs Streamable HTTP (remote).
- Connect **multiple MCP servers** to a single LLM agent.

# What is MCP?

- **Model Context Protocol** — an open standard for tool integration.
- Server advertises tools with JSON Schema inputs.
- Client (agent) discovers tools, validates args, calls them.
- Same protocol, multiple transports — choose the right one for your use case.

# Two ways to connect to an MCP server

- **Local stdio** — the client **spawns** the server as a child process and pipes JSON-RPC over stdin/stdout. No port, no URL — the client starts and stops the server automatically. This is how **Cursor** and **Claude Desktop** work.
- **Remote HTTP** — the server runs independently at a URL (e.g. `127.0.0.1:8000/mcp`). You start it yourself. For **shared team servers**, cloud deployment, cross-network access.
- Same JSON-RPC protocol, same tool schemas — different wire format.
- One-line change: `server.run()` vs `server.run(transport="streamable-http")`.

# Tool discovery response

```
{
  "tools": [
    {
      "name": "lookup_users",
      "description": "Look up users by role",
      "inputSchema": {
        "type": "object",
        "properties": {
          "role": {"type": "string"}
        }
      }
    }
  ]
}
```

- → JSON Schema describes what each tool accepts
- → Agents use this to validate before calling



# Building a local MCP server

MCPServer: decorators, schemas, and transports

# Building a local MCP server with MCPServer

```
from mcp.server.mcpserver import MCPServer

server = MCPServer("API Tools")

@server.tool()
def lookup_users(role: str | None = None) -> str:
    """Look up users by role."""
    results = [m for m in users if ...]
    return json.dumps(results)

server.run()  # stdio transport
```

- → MCPServer (formerly FastMCP) generates schemas from type hints + docstrings
- → server.run() listens on stdin — the client spawns and manages this process



# Demo — Explore ship server (stdio)

Switch to terminal: python demo/demo.py → Part 2



# Exploring a remote MCP server

Streamable HTTP: the same protocol over a URL

# Streamable HTTP transport

- Server runs independently at a URL — like any web service.
- Same JSON-RPC protocol, same tool schemas — just a different wire format.
- Use **streamable_http_client** to connect from Python.
- Ideal for **shared team servers**, cloud deployment, cross-network access.
- The server doesn't need to be rewritten — one-line change to the transport.



# Demo — Explore navigation server (HTTP)

Switch to terminal: python demo/demo.py → Part 3 (first run python http_server.py in a separate terminal)



# Connecting an LLM to MCP

From tool discovery to agent loop

# 🔗 Wiring MCP servers to an LLM

- **Discover** tools from each server with `session.list_tools()` — merge them into one list.
- **Convert** MCP schemas to the OpenAI `tools` parameter format.
- **Route** calls: build a map of `tool_name → session` so you know which server to call.
- When the LLM requests a tool call, **look up** the right session and call `session.call_tool(name, args)`.
- The LLM doesn't know or care which server a tool lives on — routing is your agent's job.



# Demo — Multi-server LLM agent

Switch to terminal: python demo/demo.py → Part 4 (http_server.py still running)



# Putting it all together

Summary and exercises

# Summary

### MCP is discovery plus a schema

*The server advertises tools. The agent validates arguments and calls them.*

### Schema is documentation

*If the schema is wrong, the agent will call it wrong.*

### stdio locally, HTTP remotely

*A local process uses stdio. A remote server uses Streamable HTTP.*

### Fail soft, and sanitise inputs

*Return JSON errors, never crash. Never trust filenames, URLs, or queries.*

# MCP servers in the wild

- **Official reference servers** — GitHub (repos, PRs, issues), PostgreSQL (natural-language queries), Filesystem (ships with Claude Desktop), Brave Search.
- **Popular community servers** — Playwright (browser automation), Memory (vector + graph), Fetch (HTML extraction), SQLite.
- **Enterprise first-party** — Slack, Stripe, Sentry, Snowflake, Salesforce, Databricks — OAuth 2.1 auth, audit logging, rate limits.
- **Discovery** — MCP Registry indexes 4 000+ servers; Smithery tracks installs. Browse at github.com/modelcontextprotocol/servers.

# Exercises

01 — Local server: build a stdio MCP server for power grid management

02 — HTTP server: build an HTTP MCP server for the science lab

03 — Ship docs: build an MCP server that serves markdown documentation as resources



# Module 3 — Complete

Next: GenAI strategies